

Лабораторная работа №1

Студент: Любимов.К.А.

Группа: ИА332

1. Написать OpenMP программу численного интегрирования:

- произвольной подинтегральной функции $f()$;
- для заданного интервала интегрирования $[a; b]$;
- методом Симпсона с равномерным разбиением интервала;
- с заданной априорной точностью $\epsilon = 1E-6$ (правило Рунге).

$$f(x) = \frac{\sin(x+2)}{0.4+\cos(x)}, \quad a = -1, \quad b = 1$$

2. Провести экспериментальный анализ масштабируемости программы при числе процессов 2, 4, 16, 32, 64 (построить график).

3. Оформить результаты в виде отчета

Для задачи был написан код на C++, представленный в [репозитории](#). Была написана программа численного интегрирования для заданной функции с заданным интервалом интегрирования, методом Симпсона и заданной априорной точностью 1E-6(по правилу Рунге).

OpenMP был внедрен в цикл для подсчета суммы площадей всех отрезков на которые мы делили наш интеграл. А именно:

```
double simpson(double a, double b, int n)
{
    double sum = 0.0;
    double step = (b - a) / n;

    sum += f(a);
    sum += f(b);
    #pragma omp parallel for reduction(+ : sum)
    for (int i = 1; i < n; i++)
    {
        double x = a + i * step;
        if (i & 1)
            sum += 4 * f(x);
        else
            sum += 2 * f(x);
    }
}
```

```
    return sum * step / 3;  
}
```

Здесь строка `#pragma omp parallel for reduction(+ : sum)` отвечает за чтобы:

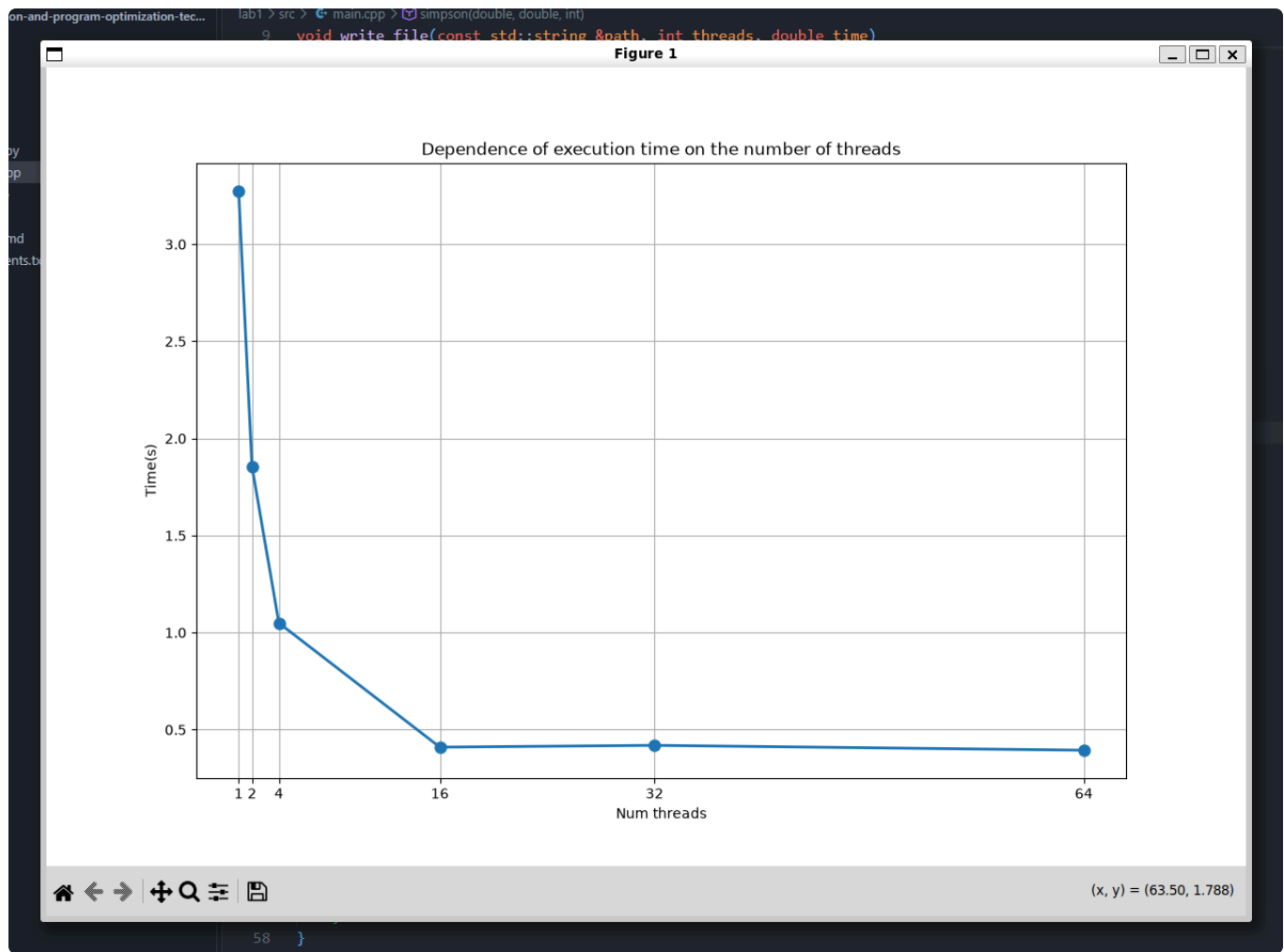
1. Инициализирует свою копию для каждой нити.
2. `+` делает эту копию изначально равной 0, и означает, что после всех вычисления результаты нитей будут объединены в одну переменную `sum`.
3. Получается что мы разделяем весь цикл `for` на кол-во нитей равномерно. Пример: есть цикл от 0 до 9, и у нас есть 5 нитей тогда:

- на 1 нить - итерации 0, 1
- на 2 нить - итерации 2, 3
- на 3 нить - итерации 4, 5
- на 4 нить - итерации 6, 7
- на 5 нить - итерации 8, 9

И в конце получаем сумму всех нитей

Результаты

Для получения реального профита от параллелизации было принято решение, не использовать Рунге, а выставить большое число Шагов разбиения в самой функции Симпсона, чтобы программа работала намного дольше. И получаем такой график:



Видно что после 16 нитей прирост незначительный. Есть сильный прирост по времени от 2 и 4 нитей. Если посчитать эффективность одной нити то получится такая таблица:

нитей	время, с	S(p)	E(p)
1	3.227	1.00	100%
2	1.711	1.89	94%
4	1.063	3.04	76%
16	0.411	7.84	49%
32	0.402	8.02	25%
64	0.384	8.41	13%